

Vue 3: computed & watch 활용하기

반응형 데이터 위에서 계산된 값과 변화 감지 패턴



Computed

템플릿을 깔끔하게 유지하고 계산 로직을 최적화하기



Watch

데이터 값이 변할 때 반응하는 사이드 이펙트 구현하기

오늘의 학습 목표

지난 시간에 배운 ref와 reactive를 바탕으로, 더 강력한 반응형 로직을 구축하는 방법을 배웁니다.

```
● ● ● state.js
// Reactivity Core
const state = ref(0)
| + computed(() => ...)
| + watch(state, ...)
```



템플릿 클린업 (Computed)

복잡한 계산식을 템플릿에서 제거하고, computed를 활용해 가독성 높고 재사용 가능한 코드로 분리합니다.



변화 감지 및 대응 (Watch)

watch를 사용하여 데이터 값이 변경될 때 API 호출이나 로깅 같은 부가 동작을 직접 구현합니다.



실전 프로젝트 실습

실전형 예제를 직접 코딩하며 실습을 진행합니다.

Remind: ref vs reactive

반응형 상태(Reactivity State)를 만드는 가장 기초적인 두 가지 방법

1

ref()

Primitives

숫자, 문자열, 불리언, 객체, 배열 (value로 접근해야 함)

```
const count = ref(0)
```

2

reactive({ ... })

Objects

여러 필드를 가진 객체 상태를 만들 때 사용합니다. 속성에 바로 접근 가능합니다.

```
const form = reactive({ price: 1000, qty: 2 })
```

Reactivity Core

Vue는 이 값들을 감시(Track)하고 있다가, 값이 바뀌면 화면을 자동 갱신(Trigger)합니다.

State Change

DOM Update

오늘 우리가 할 일

State

computed

watch

Computed 계산된 속성

computed는 상태(State)에 의존하여 자동으로 계산되는 '파생된 값'을 위한 전용 공간입니다.

언제 사용하나요?

템플릿이 복잡해질 때

HTML 안에 복잡한 연산식(예: 필터링, 정렬, 계산)이 들어가면 가독성이 떨어집니다.

중복 로직 제거

여러 곳에서 동일한 계산된 데이터를 사용할 때, 로직을 한 곳으로 모을 수 있습니다.

기본 사용법

script setup

JAVASCRIPT

```
import { computed } from 'vue'

// 계산 로직 정의
const double = computed(() => {
  return count.value * 2
})
```

TEMPLATE

```
<!-- 변수처럼 사용 -->
<div> {{ double }} </div>
```

💡 함수 호출() 없이 변수명만 씁니다.

핵심 기능: 지능형 캐싱 (Caching)

dependency 변경 시에만 실행

Vue는 내부적으로 의존성(여기선 count)을 추적합니다. count 값이 바뀌지 않으면, 아무리 많이 호출해도 다시 계산하지 않고 저장된 값을 반환합니다.

count 변경

화면 리렌더링

재계산

캐시된 값 사용

Computed 예제: doubleCount & total

Logic (Script)

App.vue

Template

```
import { ref, reactive, computed } from 'vue'

// 1. 카운터 예제 (ref)
const count = ref(0)
const doubleCount = computed(() => count.value * 2)

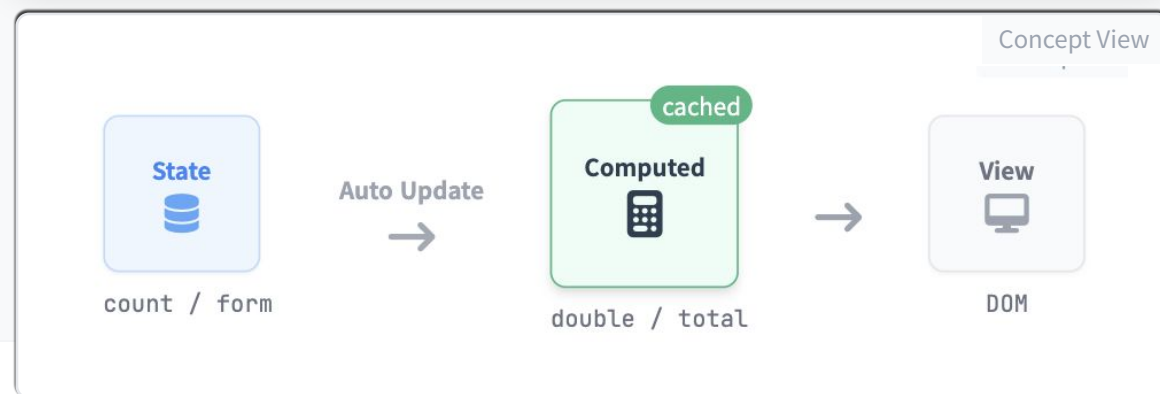
// 2. 금액 계산 예제 (reactive)
const form = reactive({
  price: 10000,
  quantity: 1,
})

// 파생된 값: 가격 * 수량
const total = computed(() =>
  form.price * form.quantity
)
```

```
<p>
  doubleCount: {{ doubleCount }}
</p>

<p>
  총액:
  <strong>
    {{ total.toLocaleString() }} 원
  </strong>
</p>
```

Tip: computed는 의존하는 반응형 상태가 변경될 때만 재계산됩니다.



직접 확인하는 실습 가이드

작성한 코드가 브라우저에서 어떻게 동작하는지 직접 테스트해봅니다. 의존성 추적과 자동 계산 과정을 눈으로 확인하세요.

KEY TAKEAWAY

"단순히 값을 보여주기 위한 목적이라면, Computed로 분리하여 템플릿을 깔끔하게 만드세요."



카운터 반응성 확인

01

Action

브라우저 화면의 +1 / -1 버튼을 클릭해보세요.

Check

count와 doubleCount가 동시에 변경됩니다.

의존성(count)이 변하면 파생 값(doubleCount)도 즉시 재계산됩니다.



쇼핑몰 총액 자동 계산

02

Action

가격 입력창과 수량 입력창의 숫자를 변경해보세요.

Check

입력 즉시 total (가격 × 수량)이 갱신됩니다.

템플릿 내의 복잡한 수식이 사라지고 코드가 깔끔해졌습니다.

watch 값 변화 감지 & 대응

"값이 변하면, 무언가를 한다." 데이터의 상태 변화를 감지하여 비동기 작업이나 DOM 조작 같은 사이드 이펙트 (Side Effect)를 실행합니다.



기본 문법 (Signature)

```
watch(  
  source, // 감시 대상 (ref, computed 등)  
  (newValue, oldValue) => {  
    // 값이 변경될 때 실행할 로직  
    console.log(`변경: ${oldValue} -> ${newValue}`)  
  }  
)
```

무엇을 감시하나?

ref

computed

reactive object

주요 활용 사례



데이터 자동 저장



API 비동기 호출



유효성 검사/경고



로그 및 분석 전송

watch 예제 1: count 변화 로그 남기기

Logic (Script)

App.vue

```
import { ref, watch } from 'vue'

const count = ref(0)

// 로그를 보여줄 상태
const lastLog = ref('아직 변경 없음')

// 값이 바뀔 때마다 실행되는 '사이드 이펙트'
watch(count, (newVal, oldVal) => {
  // 예: API 호출, 로깅, 알림 등
  lastLog.value =
    `카운트가 ${oldVal} → ${newVal} 로 변경`
})
```

Tip: watch는 API 호출, 콘솔 로그 등 값이 변했을 때 수행할 '부가 동작'에 적합합니다.

Template

```
<!-- 버튼 클릭 시 count 변경 -->
<button @click="count++">+1</button>

<p class="log">
  마지막 변경 로그:
  <span>
    {{ lastLog }}
  </span>
</p>
```

Workflow

실습 가이드

위쪽 카운터 버튼(+1 / -1)을 눌러보세요.
값이 변하는 즉시 **watch** 콜백이 실행되어 로그 문구가 업데이트됩니다.



Event



Change



Watch



Effect

Watch 심화: Computed + Watch 조합

Logic (Script)

App.vue

Template (UI 분기)

```
script setup

// 1. 총액 계산 (Computed)
const total = computed(() =>
  form.price * form.quantity
)

// 2. 총액 변화 감시 (Watch)
const isExpensive = ref(false)

watch(total, (newTotal) => {
  // 10만원 이상이면 경고
  isExpensive.value = newTotal ≥ 100000
})
```

실습: 가격을 50,000원, 수량을 2개로 설정하여 경고 메시지를 확인해보세요.

```
<p>총액: {{ total }} 원</p>

<!-- 조건부 렌더링 -->

<p v-if="isExpensive" class="warning">
  지출이 큰 편입니다! 🚨
</p>

<p v-else class="ok">
  적당한 소비입니다 👍
</p>
```



총액 계산

computed



변화 감지

watch

≥ 10만



상태 변경

isExpensive

Logic Flow

computed vs watch

언제 무엇을 사용해야 할까요?

computed

계산된 값 (파생 상태)

목적

템플릿에서 보여줄 '표현을 위한 값'을 만들 때 사용

활용 예시

총액 계산 ($\text{price} \times \text{quantity}$)

필터링/정렬된 리스트 반환

조건부 클래스 바인딩용 객체 생성

VS

watch

감시자 (부수 효과)

목적

값의 변화를 감지하여 '추가 행동(Action)'을 할 때 사용

활용 예시

데이터 변경 시 자동 저장 (Auto-save)

검색어 변경 시 API 호출

DOM 직접 조작 (차트 업데이트 등)

"보여줄 값이면 computed, 뭔가 행동해야 하면 watch"

오늘의 강의 핵심 요약

- 1 반응형 상태 (ref, reactive) 정의
- 2 computed로 파생 값 계산
- 3 watch로 변화 감지 및 대응

KEY TAKEAWAY

"보여줄 값이면 **computed**, 행동해야 하면 **watch**"

CHALLENGE

직접 해보기

1. 부가세 포함 총액 계산하기

computed를 하나 더 만들어 보세요. 기존 총액에 10% 부가세를 더한 최종 결제 금액을 표시해 봅시다.

```
const finalPrice = computed(() => total.value * 1.1)
```

2. 빈 장바구니 알림 띄우기

watch를 사용해 총액이 0원이 되는 순간을 포착하세요. "장바구니가 비었습니다"라는 메시지를 화면에 띄워보세요.

```
if (newVal === 0) showEmptyMessage.value = true
```